

Architecture Approaches with Staff Hosting Variants

Approach 1: Fully On-Prem Core Application

1A. Staff Application – On-Prem Hosted

Architecture

- Entire system (UI, business logic, database, documents) hosted in company DC
- Staff access via intranet (LAN)
- External users (Agent/Policy Holder) access through DMZ + firewall
- Single monolithic application

Flow

- Staff → Intranet → App Server → On-Prem DB/NAS
- External Users → Internet → DMZ → App Server

✓ Best for tight internal latency

✗ Poor internet scalability

1B. Staff Application – Cloud Hosted (Hybrid)

Architecture

- Core application still on-prem
- Staff UI deployed in Cloud (VDI/Web App)
- Secure VPN / MPLS / ExpressRoute connects cloud → on-prem app
- External users still hit on-prem endpoint

Flow

- Staff → Cloud UI → Secure VPN → On-Prem App → DB
- External Users → Internet → DMZ → On-Prem App

✓ Better staff accessibility

✗ Network dependency affects performance & reliability

Approach 2: Cloud Hosted Monolith (No Microservices)

2A. Staff Application – On-Prem Hosted

Architecture

- Main application (monolith) hosted in Cloud (VM / App Service)

- Database & documents in Cloud (Managed DB + Object Storage)
- Staff UI runs on intranet servers
- Secure connection from on-prem staff UI → cloud backend

Flow

- Staff → On-Prem UI → Private VPN → Cloud Monolith
- External Users → Internet → WAF/CDN → Cloud Monolith

✅ Strong cloud scalability with controlled staff access

❌ WAN latency for staff transactions

2B. Staff Application – Cloud Hosted

Architecture

- Entire system hosted in Cloud
- Separate Staff UI deployed in private subnet
- Agents & Policy Holders use public endpoints
- Staff uses VPN / Private Endpoint

Flow

- Staff → VPN → Cloud UI → Cloud Monolith → DB
- External Users → Internet → WAF/CDN → App

✅ Balanced performance & reliability

✅ Simplest ops model for most enterprises

Approach 3: Cloud Microservices (Containerized)

3A. Staff Application – On-Prem Hosted

Architecture

- Backend: Microservices on Kubernetes (AKS/EKS)
- Per-service databases
- Staff UI on-premises
- API Gateway exposed via private network

Flow

- Staff → On-Prem UI → Private Link → API Gateway → Services
- External Users → Internet → WAF/CDN → API Gateway

- ✓ Strong backend scalability
- ✗ Staff experience sensitive to network issues

3B. Staff Application – Cloud Hosted (Recommended)

Architecture

- Fully cloud-native
- Separate UIs:
 - Internet UI (Agent/Policy Holder)
 - Private UI (Staff)
- Zero-trust, token-based security
- Event-driven workflows for claims/policy approvals

Flow

- Staff → VPN / Zero Trust → Staff UI → API Gateway → Services
- External Users → Internet → WAF → Gateway

- ✓ Highest performance, reliability, and fault isolation
- ✓ Ideal for future growth

Performance & Reliability Trade-off (Staff Hosting Focus)

Dimension	On-Prem Core + Staff On-Prem	On-Prem Core + Staff Cloud	Cloud Monolith + Staff On-Prem	Cloud Monolith + Staff Cloud	Cloud Microservices + Staff On-Prem	Cloud Microservices + Staff Cloud
Staff Latency	★★★ (LAN speed)	★★ (VPN dependent)	★★ (WAN latency)	★★★	★★	★★★
External User Performance	★	★	★★★	★★★	★★★	★★★
Scalability	★	★	★★	★★	★★★	★★★
Fault Isolation	★	★	★★	★★	★★★	★★★
Availability (HA/DR)	★★	★★	★★★	★★★	★★★	★★★

Network Dependency	Low	High	Medium	Low	High	Low
Operational Complexity	Medium	High	Medium	Medium	High	High
Recovery from Failures	Manual / Slower	Slower	Faster (Cloud)	Faster	Fast but complex	Fastest & granular
Best Use Case	Strict compliance, low growth	Transitional hybrid	Lift-and-shift	General enterprise	Partial cloud adoption	Large scale, long-term platform

Architectural Recommendation Summary

Scenario	Best Choice
Minimal change, compliance-driven	Approach 1A
Gradual cloud adoption	Approach 1B → 2A
Balanced cost & performance	Approach 2B 
Long-term scalability & resilience	Approach 3B   